

INFORME: TPI PROGRAMACIÓN 2

Sistema de Gestión de Usuarios y Credenciales

Tecnologías utilizadas: Java, MySQL, JDBC, Patrón DAO

Integrante: Walter Joel Martinez Velásquez

1. Elección del Dominio y Justificación

Para este Trabajo Práctico se seleccionó la relación de dominio:

Usuario → CredencialAcceso

Justificación

La separación entre la entidad **Usuario** (datos de perfil, contacto y estado) y **CredencialAcceso** (datos de autenticación sensibles) constituye una práctica fundamental en el diseño de software seguro y escalable.

Beneficios de esta separación:

- **Seguridad:** Permite aislar la información crítica, como hashes de contraseñas y salts, facilitando la implementación de políticas de seguridad sin exponer datos personales.
 - **Escalabilidad:** Si en el futuro se requiere modificar el mecanismo de autenticación (OAuth, JWT, Tokens, etc.), únicamente se vería afectada la entidad CredencialAcceso.
 - **Cumplimiento de la consigna:** Representa una relación 1 a 1 fuerte, donde cada usuario posee un único conjunto de credenciales asociado exclusivamente a él.
-

2. Diseño y Modelado (UML)

El sistema fue diseñado siguiendo principios de orientación a objetos, utilizando herencia y composición para estructurar las entidades.

2.1 Modelo de Clases

Se implementó una clase abstracta denominada **Base**, encargada de centralizar atributos comunes de persistencia:

- **id:** identificador único.
- **eliminado:** indicador de baja lógica.

Usuario (Entidad A)

Hereda de Base y contiene:

- username
- email
- activo
- fechaRegistro

Además, mantiene una referencia a **CredencialAcceso**, siendo la entidad propietaria de la relación.

CredencialAcceso (Entidad B)

Hereda de Base y contiene:

- hashPassword
- salt
- ultimoCambio
- userId

Aunque la tabla almacena la clave foránea, la navegación principal se realiza desde la entidad Usuario.

2.2 Relación 1 a 1 Unidireccional

Se respetó la restricción de unidireccionalidad establecida por la consigna.

La clase Usuario posee el atributo:

`private CredencialAcceso credencial;`

La clase CredencialAcceso no contiene una referencia a Usuario, evitando dependencias circulares en memoria.

3. Arquitectura por Capas

La aplicación fue estructurada mediante una arquitectura por capas para garantizar la separación de responsabilidades y favorecer el mantenimiento del código.

3.1 Capa Config

Contiene la clase:

DatabaseConnectionPool

Características:

- Implementación mediante HikariCP.
- Gestión eficiente de conexiones.
- Lectura de credenciales desde un archivo externo db.properties.

3.2 Capa Models

Contiene las entidades:

- Base
- Usuario
- CredencialAcceso

Representan directamente las tablas de la base de datos.

3.3 Capa DAO

Incluye:

- Interfaz GenericDAO.
- Implementaciones:
 - UsuarioDAOImpl
 - CredencialAccesoDAOImpl

Características:

- Uso de PreparedStatement.
- Prevención de inyección SQL.
- Métodos sobrecargados para recibir objetos Connection externos y permitir control transaccional.

3.4 Capa Service

Responsable de la lógica de negocio.

Incluye:

- UsuarioServiceImpl.
- Orquestación del registro conjunto Usuario + CredencialAcceso.
- Uso de BCrypt para el almacenamiento seguro de contraseñas.

3.5 Capa Main

Contiene:

AppMenu

Responsable de:

- Interacción con el usuario.
- Validación de entradas.
- Manejo de excepciones.

4. Persistencia y Base de Datos

La persistencia se realiza sobre una base de datos MySQL denominada:

gestion_usuarios_2

Estructura de Tablas

Tabla usuario

- id (PK, AUTO_INCREMENT)
- username
- email
- eliminado

Tabla credencial_acceso

- id (PK)
- user_id (FK UNIQUE)
- hash_password
- salt

Restricciones

- La columna user_id posee una restricción UNIQUE para garantizar la relación 1 a 1.
- Se configuró ON DELETE CASCADE para mantener la integridad referencial.

5. Manejo de Transacciones (ACID)

Uno de los puntos más relevantes del proyecto es la implementación de transacciones programáticas dentro de:

UsuarioServiceImpl.registrarUsuario()

Flujo de la Transacción

1. Obtención de una conexión desde el pool.
2. Desactivación del autocommit.
3. Inserción del Usuario.
4. Recuperación del ID generado.
5. Creación e inserción de CredencialAcceso.
6. Ejecución de commit cuando ambas operaciones son exitosas.

7. Ejecución de rollback ante cualquier excepción.

```
@Override
// Metodos sobrecargados, para poder manejar las transacciones
public void insert(Connection conn, Usuario user) throws SQLException {

    // ¡OJO! NO hay try-with-resources para la 'conn'
    // porque la recibimos de afuera y no queremos cerrarla.
    try (PreparedStatement stmt = conn.prepareStatement(INSERT_SQL, Statement.RETURN_GENERATED_KEYS)) {

        stmt.setString(1, user.getUsername());
        stmt.setString(2, user.getEmail());

        int rowsAffected = stmt.executeUpdate();
        // si no hay, no lo insertamos
        if (rowsAffected == 0) {
            throw new SQLException("No se pudo insertar el usuario.");
        }

        try (ResultSet generatedKeys = stmt.getGeneratedKeys()) {
            if (generatedKeys.next()) {
                int nuevoId = generatedKeys.getInt(1);
                user.setId(nuevoId);
            } else {
                throw new SQLException("Se inserto el usuario, pero no se pudo recuperar el ID.");
            }
        }
    }
}
```

Esto garantiza que no existan usuarios registrados sin credenciales asociadas.

6. Validaciones y Reglas de Negocio

Se implementaron validaciones tanto en la capa de servicios como en la base de datos

Contraseñas Hasheadas

No se almacenan contraseñas en texto plano

Se utiliza BCrypt para:

- Generar salts aleatorios.
- Hashear contraseñas.
- Validar credenciales durante el login.

Campos Obligatorios

Se valida que:

- username
- email
- password

no sean nulos ni vacíos antes de acceder a la capa DAO

Validación de Correo Electrónico

Se realiza una validación básica verificando la presencia del carácter: @

Baja Lógica

Los usuarios no son eliminados físicamente de la base de datos.

En su lugar se ejecuta:

UPDATE usuario

SET eliminado = TRUE

Las consultas posteriores filtran automáticamente por eliminado = FALSE para ocultar estos registros

7. Pruebas Realizadas

Las pruebas fueron ejecutadas manualmente mediante el menú de consola de la aplicación.

Caso de Prueba 1

Inicio de Sesión luego del Registro de Usuario

```
===== GESTION DE USUARIOS =====
1) Crear usuario (A + credencial B)
2) Iniciar Sesion
3) Buscar usuario por ID
4) Listar usuarios
5) Actualizar usuario
6) Eliminar usuario
7) Buscar por EMAIL (campo relevante)
8) Restaurar usuario
0) Salir
Seleccione una opción: 2
Ingrese su usuario:
pepito
Ingrese su contraseña:
pepel2345
LOGIN EXITOSO! Bienvenido! pepito
```

Se verificó:

- Inicio de sesión correcto

Caso de Prueba 2

Listado de Usuarios

```

===== GESTION DE USUARIOS =====
1) Crear usuario (A + credencial B)
2) Iniciar Sesión
3) Buscar usuario por ID
4) Listar usuarios
5) Actualizar usuario
6) Eliminar usuario
7) Buscar por EMAIL (campo relevante)
8) Restaurar usuario
0) Salir
Seleccione una opción: 4
Usuario{id = 524281, username = joel, email = joel@gmail.net, activo = true, fechaRegistro = 2026-06-18T19:43:11, eliminado = false, credencial = 500001}
Usuario{id = 524282, username = Walter, email = waltermartinez@hotmail.com, activo = true, fechaRegistro = 2026-06-18T22:00:16, eliminado = false, credencial = 500003}
Usuario{id = 524283, username = erica, email = ericam@gmail.com, activo = false, fechaRegistro = 2026-06-18T22:22:50, eliminado = false, credencial = 500004}
Usuario{id = 524284, username = Jose, email = joecoro@mail.com, activo = true, fechaRegistro = 2026-06-18T22:32:55, eliminado = false, credencial = 500004}
Usuario{id = 524285, username = pepito, email = pepito@email.com, activo = false, fechaRegistro = 2026-06-22T20:02:40, eliminado = false, credencial = 500005}

```

Se comprobó:

- Recuperación de registros
- Visualización correcta de los datos almacenados

Caso de Prueba 3

Eliminación de Usuario

```

===== GESTION DE USUARIOS =====
1) Crear usuario (A + credencial B)
2) Iniciar Sesión
3) Buscar usuario por ID
4) Listar usuarios
5) Actualizar usuario
6) Eliminar usuario
7) Buscar por EMAIL (campo relevante)
8) Restaurar usuario
0) Salir
Seleccione una opción: 6
ID a eliminar: 524285
Usuario eliminado logicamente.

```

	id	username	email	fecha_registro	activo	eliminado
▶	524281	joel	joel@gmail.net	2026-06-18 19:43:11	1	0
	524282	Walter	waltermartinez@hotmail.com	2026-06-18 22:00:16	1	0
	524283	erica	ericam@gmail.com	2026-06-18 22:22:50	0	0
	524284	Jose	joecoro@mail.com	2026-06-18 22:32:55	1	0
	524285	pepito	pepito@email.com	2026-06-22 20:02:40	0	1
*	NULL	NULL	NULL	NULL	NULL	NULL

Se verificó:

- Aplicación correcta de la baja lógica
- Actualización del campo eliminado

Caso de Prueba 4

Recuperación de Usuario

```
===== GESTION DE USUARIOS =====
1) Crear usuario (A + credencial B)
2) Iniciar Sesion
3) Buscar usuario por ID
4) Listar usuarios
5) Actualizar usuario
6) Eliminar usuario
7) Buscar por EMAIL (campo relevante)
8) Restaurar usuario
0) Salir
Seleccione una opción: 8
ID a restaurar: 524285
Usuario restaurado.
```

	id	username	email	fecha_registro	activo	eliminado
▶	524281	joel	joel@gmail.net	2026-06-18 19:43:11	1	0
	524282	Walter	waltermartinez@hotmail.com	2026-06-18 22:00:16	1	0
	524283	erica	ericam@gmail.com	2026-06-18 22:22:50	0	0
	524284	Jose	joecoro@mail.com	2026-06-18 22:32:55	1	0
	524285	pepito	pepito@email.com	2026-06-22 20:02:40	0	0
*	NULL	NULL	NULL	NULL	NULL	NULL

Se comprobó:

- Restauración de usuarios previamente eliminados
- Correcta actualización del estado lógico

8. Conclusiones y Mejoras Futuras

El desarrollo de este Trabajo Práctico permitió consolidar conocimientos sobre JDBC, DAO, arquitectura por capas y manejo transaccional.

Se logró desacoplar correctamente la lógica de negocio del acceso a datos, implementando una relación 1 a 1 robusta y segura. Informe - TPI Programacion 2.pdf

Puntos Fuertes

- Uso de HikariCP para la gestión profesional de conexiones
- Implementación de BCrypt para seguridad de credenciales
- Manejo adecuado de transacciones mediante commit y rollback
- Arquitectura desacoplada y escalable

Mejoras Futuras

1. Implementación de recuperación de contraseña por correo electrónico
2. Incorporación de roles y permisos (Administrador / Usuario)
3. Integración con APIs de autenticación modernas
4. Implementación de pruebas automatizadas unitarias e integrales

[LINK al video](#)

[LINK al repositorio](#)